#DSA_Week7

* DSA - Week - 7 Assignments *

① Unweighted shortpaths and weighted shortpaths:-

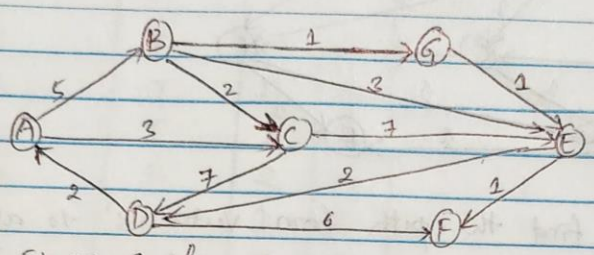
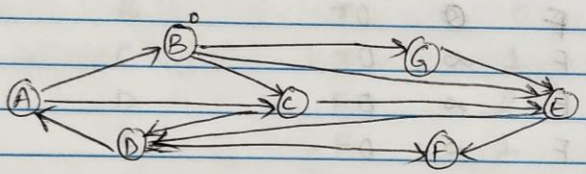


Fig: 9.82: Graph:-

Ans:- *) Finding "shortest unweighted path" from 'B' to all the other vertices:-

So, the given graph turns into

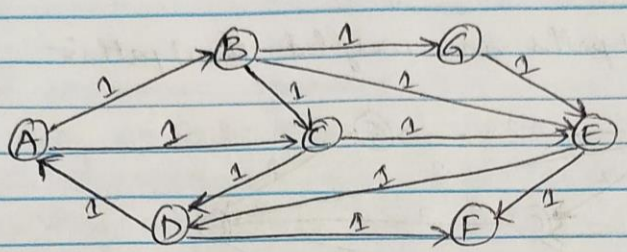


So, for the unweighted graphs we don't have any edges weights. ↑ right.
To find the shortest path, we can assign all edges weight of '1'.

→ For finding the shortest path we know that BFS is the best method to find.

→ So, here we are using Breadth First Search algorithm (method) for finding shortest path.

→ We need to use the distance table for representing this traversal of the BFS graph.



→ we need to find the path form vertex 'B' to all the other vertex's so, the tables for data change during unweighted shortest path is given below.

Vertex(v)	Known	d_v	P_v
A	F	∞	0
B	F	∞	0
C	F	∞	0
D	F	∞	0
E	F	∞	0
F	F	∞	0
G	F	∞	0

→ This is initial state.

Vertex (v)	Known	d_v	P_v
A	F	∞	0
B	T	0	0
C	F	1	B
D	F	∞	0
E	F	1	B
F	F	∞	0
G	F	1	B

→ After 'B' dequeued.

Vertex(V)	Known	d_v	P_v
A	F	∞	0
B	T	0	0
C	T	1	B
D	F	2	C
E	F	1	B
F	F	∞	0
G	F	1	B

→ Table after vertex 'c' dequeued.

Vertex (V)	known	d_v	P_v
A	F	∞	0
B	T	0	0
C	T	1	B
D	F	2	C
E	F	1	B
F	F	∞	0
G	T	1	B.

→ Table after dequeing vertex 'G'. (no change)

Vertex	known	d_v	P_v
A	F	∞	0
B	T	0	0
C	T	1	B
D	F	2	C
E	T	1	B
F	F	2	E
G	T	1	B.

→ Table after performing dequeue operation of vertex 'E'.

Vertex	known	dv	Pv
A	F	3	D
B	T	0	0
C	T	1	B
D	T	2	C
E	T	1	B
F	F	2	E
G	T	1	B.

→ Table after dequeuing Vertex 'D'.

→ Here we known all the vertices distances, so dequeuing the remaining vertexes in single step which is Vertex A & F

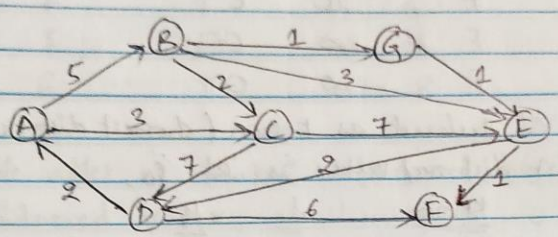
Vertex	known	dv	Pv
A	T	3	D
B	T	0	0
C	T	1	B
D	T	2	C
E	T	1	B
F	T	2	E
G	T	1	B.

→ The above table is the final table after performing all the operations;

#

* Finding the shortest weighted path from 'A' to all other vertices using Dijkstra's algorithm.

→ same as before, we need to have a distance table which holds the values from starting vertex 'A'



→ This is the weighted graph for which we need to find shortest weighted path from vertex 'A'.

Vertex	known	d_v	P_v
A	F	0	0
B	F	∞	0
C	F	∞	0
D	F	∞	0
E	F	∞	0
F	F	∞	0
G	F	∞	0

→ Initial configuration of the table using Dijkstra's algorithm.

Vertex	known	d_v	P_v
A	T	0	0
B	F	5	A
C	F	3	A
D	F	∞	0
E	F	∞	0
F	F	∞	0
G	F	∞	0

→ Table after 'A' declared as known.

#

→ Next we need to select the shortest distanced vertex so,

Vertex	Known	d_v	P_v
A	T	0	0
B	F	5	A
C	T	3	A
D	F	10	C
E	F	10	C
F	F	∞	0
G	F	∞	0

→ Table after 'C' declared as known (shortest distanced vertex).

→ The next shortest distanced vertex is 'B' so,

Vertex	Known	d_v	P_v
A	T	0	0
B	T	5	A
C	T	3	A
D	F	10	C
E	F	8	B
F	F	∞	0
G	F	6	B

→ Table after 'B' marked as known.

Vertex	Known	d_v	P_v
A	T	0	0
B	T	5	A
C	T	3	A
D	F	10	C
E	F	7	G
F	F	∞	0
G	T	6	B

→ Table after 'G' marked as known (next shortest distanced vertex).

Vertex	known	d_v	P_v
A	T	0	0
B	T	5	A
C	T	3	A
D	F	9	E
E	T	7	G
F	F	8	E
G	T	6	B

→ Table after 'E' marked as known, which is the next shortest distanced vertex.

→ Next short distanced vertex is 'F' but it has no output nodes so there won't be any changes in the graph. So I'm taking two nodes D, & F as known. which is the final step.

Vertex	Known	d_v	P_v
A	T	0	0
B	T	5	A
C	T	3	A
D	T	9	E
E	T	7	G
F	T	8	E
G	T	6	B

→ ∴ This is the final table of weighted graph using Dijkstra's algorithm.

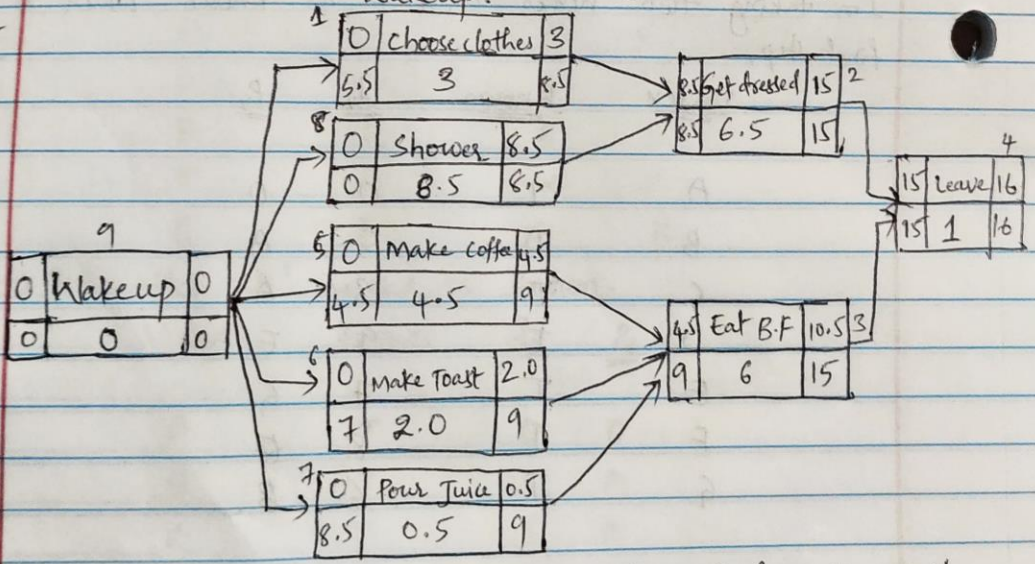
#

② Task scheduling and Critical Path :-

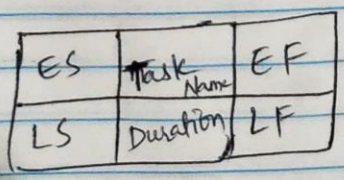
→ Consider the following problem description :

Task Number	Task Name	Duration	Dependson
1	choose clothes	3	9
2	Get dressed	6.5	1, 8
3	Eat Breakfast	6	5, 6, 7
4	Leave	1	2, 3
5	Make coffee	4.5	9
6	Make Toast	2.0	9
7	Pour Juice	0.5	9
8	shower	8.5	9
9	Wakeup.	0	-

Ans-



→ The above figure shows the Event-node graph from the table.



ES → Early start value
EF → Early finish value
LS → Late start value
LF → Late finish value.

EF = ES + Duration

→ The Early start of the "wake up" start is '0' because we are starting from that note, i.e. it is the starting node.

→ Early Finish of the first node is sum of the values of Early Start and duration. This is similar for each and every node.

→ After completing Early finish value, that value is tended towards the next pointing node.

→ If the two nodes are pointing during forward pass to the single node, then we need to take the highest values of those two. Vice-versa for the backward pass.

→ The value which is present at the end node of 'EF' is the maximum number of duration took for the task to be completed.

→ In our case total duration is '16'.

→ For the backward pass, at the end node, LF value is taken from the "EF".

→ To calculate the value of "LS = LF - Duration". for every node.

→ During the backward pass, if there are more than one node then it will take the value which is lesser.

→ In our case, at the ~~end~~ start node, we have five different values so, '0' is least from that so it takes the value of zero.

→ After completing all of these steps, we are going to find out Critical path or critical tasks.

→ The Tasks which has EF and LF as same are considered as critical tasks

→ In our case, critical tasks are : wake up, shower, Get dressed & leave.

→ These are the only tasks which has Early Finish and Late finish values same so considered as critical tasks.

→ We have one critical path. which is,

Wake up → shower → Get dressed → Leave.

∴ 9 → 8 → 2 → 4

This is the critical path.

→ The values of Earliest completion time, late completion time and slack time are in the figure as, EF, LF, & LS respectively.

#

9.52) A student needs to take a certain number of courses to graduate, and these courses have prerequisites that must be followed. Assume that all courses are offered every semester, and that the student can take an unlimited number of courses. Given a list of courses and their prerequisites, compute a schedule that requires the minimum number of semesters.

- **Give clear explanations on how this problem can be solved using graph model: What do the vertices represent? What do the edges represent?**
- **Describe the algorithm used to find the answer.**
- **Analyze the time complexity of the algorithm.**

Ans: -

To solve this problem, we can re-present the courses and their prerequisites as a directed graph. Here's a detailed explanation of the approach:

Vertices Representation: Each vertex in the graph represents a course. For example, if we have courses A, B, and C, where A has prerequisites B and C, we can represent the prerequisites using edges.

Edges Representation: The edges in the graph represent the prerequisites of each course. For example, in the graph representing the courses A, B, and C, where A has prerequisites B and C, there would be edges from vertices B and C to vertex A.

Algorithm:

1. First, we initialize a set to keep track of all the courses visited and another set to keep track of the courses that are currently being visited.
2. Then, we perform a Depth-First Search (DFS) on each course. In the DFS, we start by marking the current course as being visited.
3. After that, we will visit all the prerequisites of the current course. If any of the prerequisites are currently being visited, it means there is a cycle in the graph, which means it's impossible to complete the prerequisites for all the courses in the given number of semesters. In this case, we return false to indicate that the prerequisites can't be met.
4. If none of the prerequisites have a cycle, we remove the current course from the set of courses that are currently being visited and add it to the set of courses visited.
5. After visiting all the courses, if there are any courses that were not visited, it means they have not been met. In this case, we return false to indicate that the prerequisites can't be met.
6. If all the courses were visited, we return true to indicate that it's possible to meet the prerequisites for all the courses in the given number of semesters.

Time complexity: O (V + E):
The time complexity of this solution is O (V + E), where V is the number of vertices (courses) and E is the number of edges (prerequisites). This is because in the worst case, we may need to visit each course and each of its prerequisites.

9.53) The object of the *Kevin Bacon Game* is to link a movie actor to Kevin Bacon via shared movie roles. The minimum number of links is an actor's *Bacon number*. For instance, Tom Hanks has a Bacon number of 1; he was in *Apollo 13* with Kevin Bacon. Sally Field has a Bacon number of 2, because she was in *Forrest Gump* with Tom Hanks, who was in *Apollo 13* with Kevin Bacon. Almost all well-known actors have a Bacon number of 1 or 2. Assume that you have a comprehensive list of actors, with roles,[3] and do the following:

a. Explain how to find an actor's Bacon number.

b. Explain how to find the actor with the highest Bacon number.
c. Explain how to find the minimum number of links between two arbitrary actors.

- **Give clear explanations on how this problem can be solved using graph model: What do the vertices represent? What do the edges represent?**
- **Describe the algorithm used to find the answer.**
- **Analyze the time complexity of the algorithm.**

Ans: -

a)

→ Start with the actor whose Bacon number is to be determined as the source vertex.
→ Perform a breadth-first search (BFS) traversal of the graph, considering the shared movie roles as edges.
→ During the BFS traversal, maintain a level or distance from the source vertex.
→ When the BFS traversal reaches Kevin Bacon, the distance or level from the source vertex to Kevin Bacon will be the actor's Bacon number.

b)

→ Perform a BFS traversal of the graph starting from each actor as the source vertex.
→ Keep track of the maximum distance or level reached during each traversal.
→ The actor with the highest distance or level is the one with the highest Bacon number.

c)

→ To find the minimum number of links between two arbitrary actors:
→ Perform a BFS traversal of the graph starting from one of the actors as the source vertex.
→ During the BFS traversal, maintain a level or distance from the source vertex.
→ If the other actor is encountered during the traversal, the distance or level from the source vertex to that actor is the minimum number of links between the two actors.

In the graph model for the Kevin Bacon game: -

Vertices: Each vertex represents an actor.

Edges: The edges represent shared movie roles between actors. If two actors have appeared in the same movie, there will be an edge connecting their respective vertices.

The algorithm used to find the Bacon number, the actor with the highest Bacon number, or the minimum number of links between two actors is a BFS traversal on the graph.

- → Start with the actor whose Bacon number is to be determined.
- → Initialize a queue and enqueue the starting actor with a Bacon number of 0.
- → Perform a BFS from the starting actor, visiting actors connected through shared movie roles.
- → As you visit each actor, assign them a Bacon number one greater than the current actor's Bacon number.
- → Continue the BFS until you reach Kevin Bacon or exhaust all possible connections.

The Bacon number of the starting actor is the final Bacon number assigned to Kevin Bacon in this process.

Algorithm to Find the Minimum Number of Links Between Two Arbitrary Actors:

To find the minimum number of links between two arbitrary actors, you can use a shortest path algorithm, such as Dijkstra's or Breadth-First Search. In this case, you would consider the shared movie roles as edges with unit weight. Here's how to do it:

- → Create a graph with actors as vertices and share movie roles as weighted edges (weight = 1).
- → Choose one of the actors as the source and the other as the target.
- → Use a shortest path algorithm (e.g., Dijkstra's or BFS) to find the shortest path between the source and target actors in the graph.
- → The length of the shortest path will give you the minimum number of links (shared movie roles) between the two actors.

The time complexity of the BFS algorithm is O (V + E), where V is the number of vertices (actors) and E is the number of edges (shared movie roles). The algorithm visits each vertex and edge once during the BFS traversal. The time complexity can be further improved if the graph is represented using an adjacency list instead of an adjacency matrix.

---

3). (All-Pairs Shortest Paths & Dynamic Programming) Implement the All-pairs shortest path algorithm with dynamic programming. Use it to find the shortest weighted path from every vertex to every other vertex in Figure 9.82. Print out Matrix from D1 to D7.

Ans: For this code, I have taken the values of matrix for each node to node.

That is if we have node weight of 5 between node A and B then I have entered that value at place of row (A) to column (B) position as well as column (A) to row (B) position as well.

I have not entered the values as **All pairs shortest path method**, because I can't enter two matrices in one program so, I am just confused between what values to use so I just used those values.

The output is shown as below:

0.00 5.00 3.00 2.00 4.00 5.00 5.00
5.00 0.00 2.00 4.00 2.00 3.00 1.00
3.00 2.00 0.00 5.00 4.00 5.00 3.00
2.00 4.00 5.00 0.00 2.00 3.00 3.00
4.00 2.00 4.00 2.00 0.00 1.00 1.00
5.00 3.00 5.00 3.00 1.00 0.00 2.00
5.00 1.00 3.00 3.00 1.00 2.00 0.00